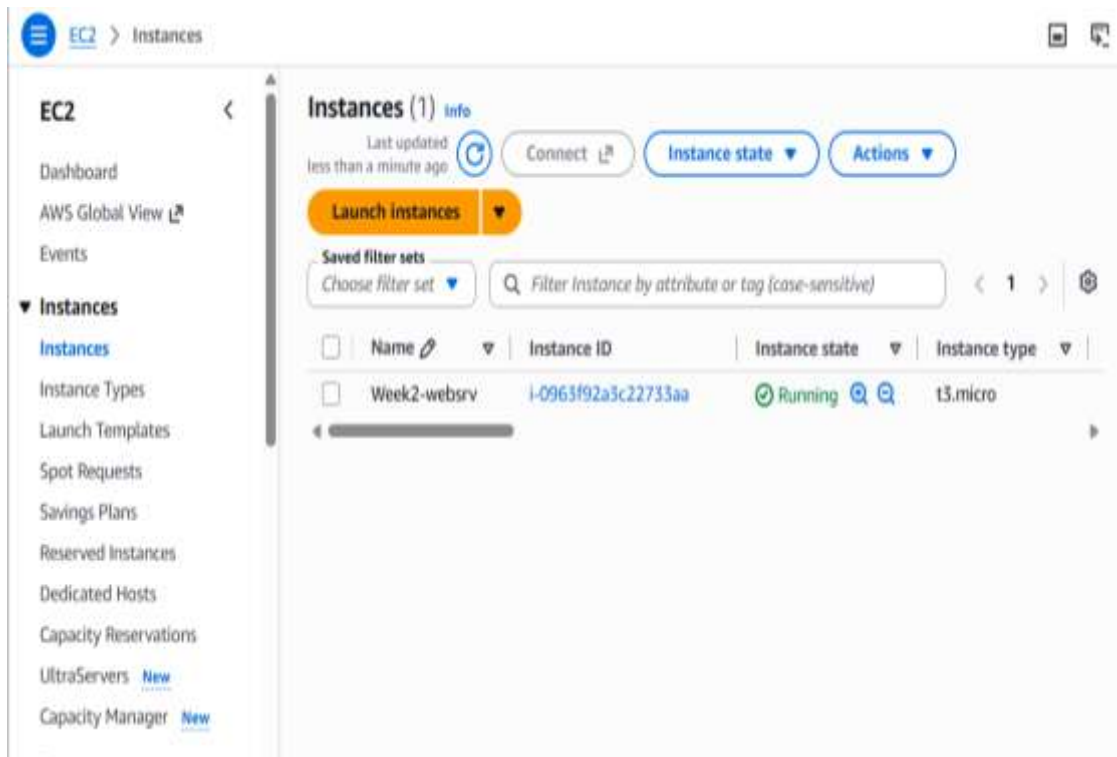


Assignment+ Mini Project: EC2 Web Server and S3 Static Website Hosting

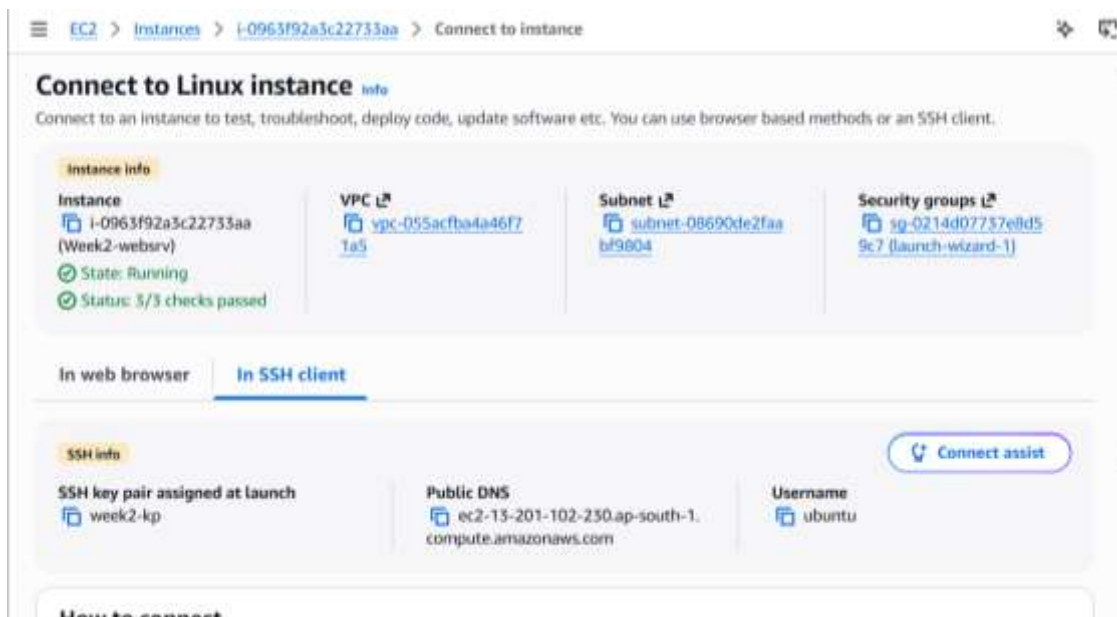
1. Launched and Connected to an EC2 Ubuntu Instance

Steps:

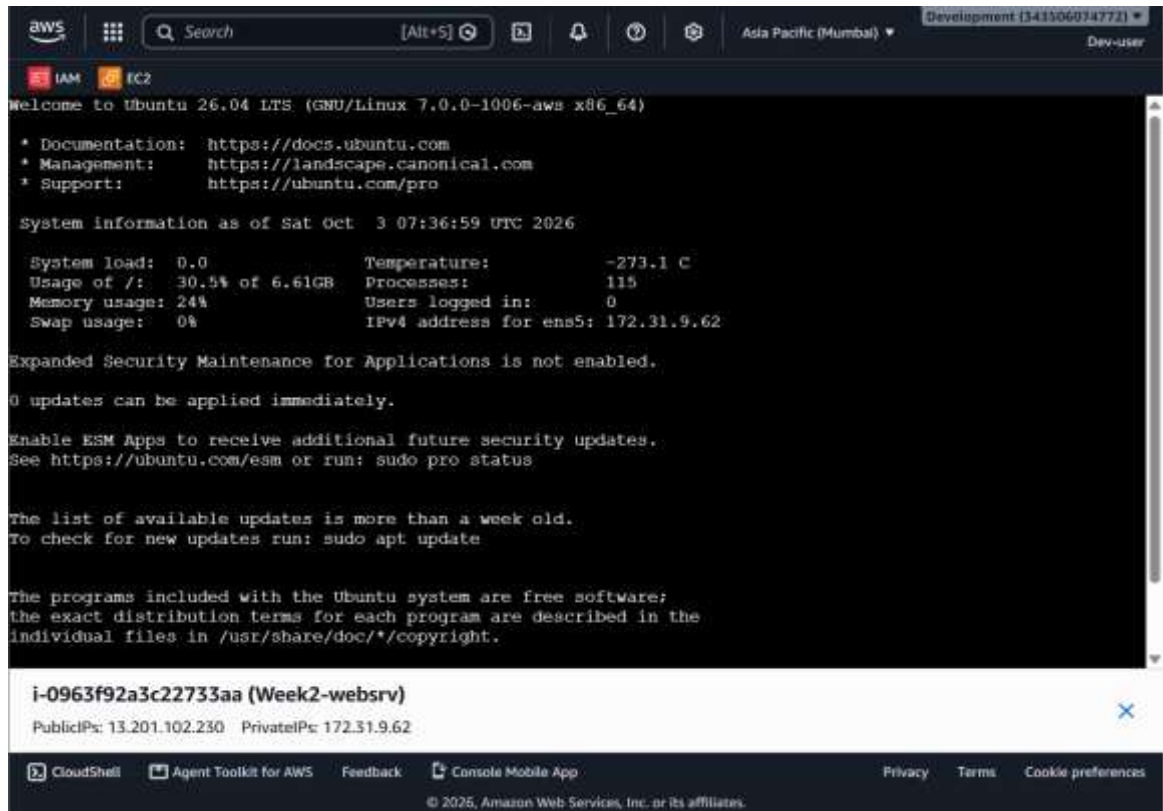
- Launched an EC2 instance named "Week2-websrv" using Ubuntu 26.04 LTS, instance type t3.micro (free-tier eligible).
- Created a new key pair (week2-kp, RSA, .pem format) for secure access.
- Configured the security group to allow SSH (port 22) and HTTP (port 80) inbound traffic.
- Connected to the instance using EC2 Instance Connect (browser-based SSH), logged in as the "ubuntu" user.



EC2 instance "Week2-websrv" running (t3.micro, Ubuntu)



Connect to Linux instance screen showing instance details, key pair, and public DNS



Successfully connected via EC2 Instance Connect — Ubuntu terminal session

2. Installed and Configured a Web Server (Nginx)

Steps:

- Updated the package list: `sudo apt update`
- Installed Nginx: `sudo apt install nginx -y`
- Verified the service was active and running: `sudo systemctl status nginx`
- Tested the default Nginx page over HTTP using the instance's public IP.
- Edited `/var/www/html/index.html` using `nano` to replace the default page with a custom message.
- Saved the file, refreshed the browser, and confirmed the custom page loaded correctly.

```
sudo apt update
```

```
sudo apt install nginx -y
```

```
sudo systemctl status nginx
```

```
sudo nano /var/www/html/index.html
```

```
aws | IAM | EC2
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-9-62:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-10-03 07:50:09 UTC; 31s ago
     Invocation: 26a96ace2c146e99838556a6cb697f2
       Docs: man:nginx(8)
    Process: 1811 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited
, status=0/SUCCESS)
    Process: 1813 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 1842 (nginx)
     Tasks: 3 (limit: 627)
    Memory: 3M (peak: 7.3M)
       CPU: 45ms
    CGroup: /system.slice/nginx.service
            └─1842 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
               └─1845 "nginx: worker process"
                  └─1846 "nginx: worker process"

Oct 03 07:50:09 ip-172-31-9-62 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server:
Oct 03 07:50:09 ip-172-31-9-62 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server:
lines 1-18/18 (END)
```

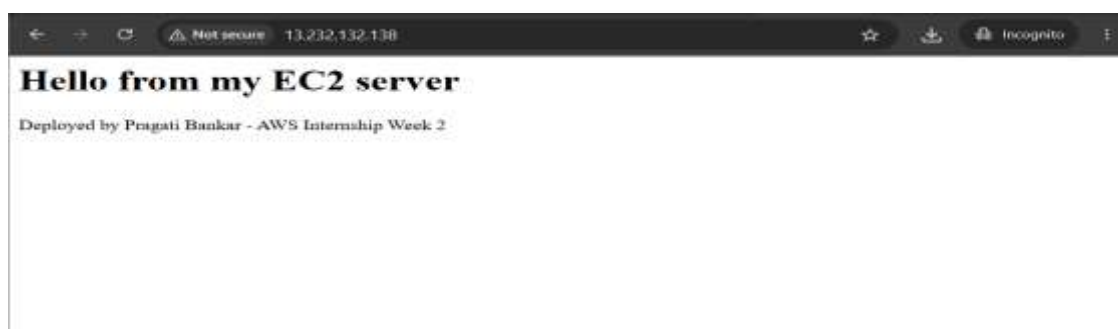
i-0963f92a3c22733aa (Week2-websrv)

Public IPs: 13.232.132.138 Private IPs: 172.31.9.62

CloudShell Agent Toolkit for AWS Feedback Console Mobile App Privacy Terms Cookie preferences

© 2026, Amazon Web Services, Inc. or its affiliates.

Nginx installed and running — "Active: active (running)" confirmed via systemctl



Custom page successfully served by Nginx over HTTP, replacing the default welcome page

3. Created an S3 Bucket and Hosted a Static Website

Steps:

- Created a new S3 bucket named "pragati-port-2026" in the Asia Pacific (Mumbai) region.
- Unchecked "Block all public access" and acknowledged the warning, since static website hosting requires public read access.
- Enabled Static Website Hosting under the bucket's Properties tab, with index.html set as the index document.
- Uploaded a multi-section portfolio website (index.html) covering summary, skills, projects, experience, and education.
- Added a bucket policy granting public s3:GetObject access so the website could be viewed by anyone.
- Tested the live site using the S3 bucket website endpoint (not the regular object URL).

Bucket policy used:

```
{
  "Version": "2012-10-17",
  "Statement": [
```

```

{
  "Sid": "PublicReadGetObject",
  "Effect": "Allow",
  "Principal": "*",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::pragati-port-2026/*"
}
]
}

```

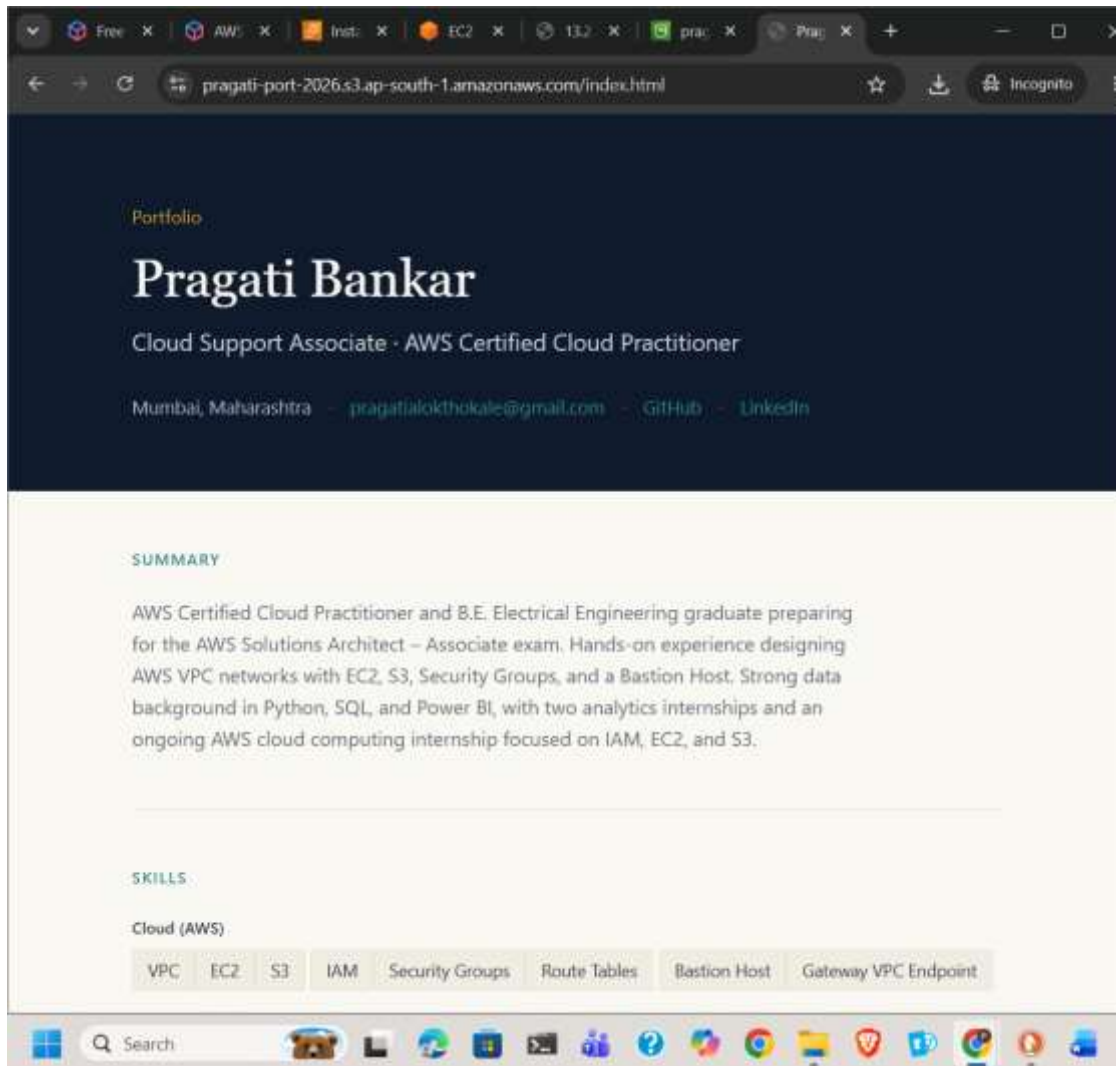
The screenshot shows the Amazon S3 console's 'Buckets' page. Under the 'General purpose buckets' tab, a single bucket named 'pragati-port-2026' is listed in the Asia Pacific (Mumbai) ap-south-1 region, created on October 3, 2026. Below the bucket list, there are sections for 'Account snapshot' and 'External access summary', both updated daily. The footer includes links to CloudShell, Agent Toolkit for AWS, and other resources.

Successfully created s3 bucket name is pragati-port-2026

The screenshot displays the 'Upload: status' page in the Amazon S3 console. A summary box indicates that the upload to the destination 's3://pragati-port-2026' was successful, with 1 file (8.7 KB) uploaded at 100.00%. Below this, the 'Files and folders' section shows a table with one entry: 'index.html', which is a text/html file of 8.7 KB, uploaded successfully.

Name	Folder	Type	Size	Status	Error
index.html	-	text/html	8.7 KB	Succeeded	-

index.html uploaded successfully to the S3 bucket (8.7 KB)



Live static website hosted on S3, accessed via the bucket website endpoint

Live website URL:

<http://pragati-port-2026.s3-website.ap-south-1.amazonaws.com>

Learning Outcome

This week gave me hands-on experience with two very different ways of serving content from AWS: a traditional EC2 virtual server running Nginx, where I manage the operating system and the web server myself, versus S3 static website hosting, where AWS serves the files directly with no server to manage at all. I also learned that S3 static website hosting uses a distinct "website endpoint" URL, separate from a regular object URL, and that it depends on the same two layers of public access control (Block Public Access and a bucket policy) that I worked with in Week 1. Troubleshooting the unexpected IAM permissions boundary error was a useful reminder that cloud environments can have account-level restrictions beyond what is visible on a single user's policy list, and that working around such issues methodically (checking as root, then narrowing down) is a normal part of cloud support work.